UNIVERSITA' DEGLI STUDI DI NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

Tesi di Laurea Magistrale in Ingegneria Informatica

Comparing Quality of Human-written and AI-generated Code

Anno Accademico 2024/2025

Relatore
Ch.mo prof. Pietro Liguori
Correlatore
Ing. Cristina Improta

Candidato
Andrea Scognamiglio
matr. M63001431

Abstract

The increase in the use of LLMs for code generation, is leading towards a new evolution of the programmer's role: from primary person responsible for the work to supervisor of the generated code. Furthermore all static and dynamic analysis tools, but also tools like compilers, are based on the viewpoint of human errors. With the advent of LLMs like DeepSeek Coder and Qwen, the focus shifts to understanding what are the defects and vulnerabilities introduced into real projects by AI generated code. In this thesis we show a methodology that allows performing a static analysis of AI generated code to compare its results in terms of quality with human code. To obtain the results, we analyzed 280518 snippets in C language extracted from real projects of human code. For the static analysis of defects, it is right to observe that such results depend on the language (C) and the specific tool used, therefore in order to make the results comparable, we performed a 1:1 mapping between each checker and the Defect Type categories of the Orthogonal Defect Classification (ODC). ODC is a standard approach to classify software defects into Defect Type Categories. Each category

is mutually exclusive with the others, so a defect could be classified in only one category. This approach helps programmers to better understand what kind of defects the code is affected, so that a targeted intervention could be done. For the analysis of vulnerabilities we have identified the Common Weakness Enumerations from Mitre, while for the complexity analysis, we have calculated existing metrics such as Number Of Line Code (NLOC) and Cyclomatic Complexity Number (CCN). The results obtained show that human code is syntactically richer and more complex compared to the AI generated one, which in turn turns out to be much more vulnerable in terms of security.

Contents

Abstract **i**

1 Introduction **1**

1.1 Thesis Contribution 4

2 Related Works **7**

3 Methodology **14**

3.1 Dataset 14

3.1.1 AI generated Dataset 16

3.1.2 Cleaning AI Dataset 19

3.2 Defect Analysis 22

3.2.1 Clang-tidy v.21.1.0 23

3.2.2 Orthogonal Defect Classification (ODC) 24

3.3 Vulnerabilities Analysis 27

3.3.1 Semgrep 28

3.4 Complexity Analysis 32

4 Experimental Results **37**

4.0.1 Defects Assessment 37

4.0.2 Vulnerabilities Assessment 47
4.0.3 Complexity Assessment 53

5 Conclusion **54**
5.1 Future Work 56

Chapter 1

Introduction

The rise of AI-assisted programming has led to a major transformation in software engineering, reshaping not only the way code is produced, but also the nature of its authorship [5]. Large Language Models (LLMs) such as ChatGPT [24], Qwen 2.5 coder [11], and DeepSeek-Coder [9] now go far beyond simple code completion or snippet generation: they are capable of producing full functions, classes, and even complete modules with minimal human involvement. This shift has gradually moved programming from a human-driven design practice toward a model-driven paradigm [18]. Although this development offers clear benefits in terms of efficiency and accessibility, it simultaneously introduces pressing concerns about the accuracy, security, and overall quality of machine-generated software. Importantly, this transformation cannot be reduced to an incremental improvement in development tools—it constitutes a structural redefinition of the dis-

cipline itself. The responsibilities of developers are evolving from coding and design toward overseeing, refining, and contextualizing model output, effectively shifting their role from creators to curators. As AI-generated code becomes more widespread, distinctive patterns emerge: it often appears shorter, highly pattern-oriented, and stylistically uniform, reusing common structures such as utility wrappers and boilerplate code. These outputs frequently make their way into production systems, open-source projects, and developer workflows with limited manual inspection. Consequently, AI-generated code is influencing the stylistic, structural, and quality standards that traditionally define software engineering. However, current research only partially captures the implications of this phenomenon. Most studies evaluate AI-generated programs alone, focusing on correctness or benchmark performance [38], [20], or examining selective dimensions of quality [16], [37] and security [33], [7]. Few works provide systematic comparisons between AI-generated and human-written code, which limits our understanding of how the two differ in terms of structural complexity, defect profiles, or security vulnerabilities. Even fewer employ a standardized framework for quality assessment, leading to fragmented results that are difficult to compare between studies due to variations in programming languages, tools, and evaluation metrics. This gap raises a fundamental issue: conventional quality assessment frameworks were designed with human-written code in mind, grounded in assumptions

about human reasoning, common errors, and review processes. In contrast, AI-generated code introduces novel features—such as structural redundancy, model-specific vulnerability patterns, and even hallucinations [14]. It remains unclear whether existing taxonomies can adequately capture these phenomena or whether they must be adapted and extended to reflect the characteristics of AI-driven development.

In this work, a large-scale study is presented comparing AI-generated and human-written code (for C language snippets), analyzing through static analysis tools, three key aspects of software quality: defects, security vulnerabilities and complexity. The models used to create the dataset are Qwen 2.5 Coder [11] and DeepSeek Coder [9]. As mentioned earlier, there is still no standardized methodology for analyzing these effects. Existing works typically examine quality and security in isolation, or they rely on non-standardized tools, which limits the comparability of their findings. For this reason, to standardize comparisons between tools and languages, this work adopts the Orthogonal Defect Classification (ODC) framework introduced by Chillarege et al. [2]. This methodology enables defects to be categorized into independent categories such as assignment, algorithm, interface etc. thereby mapping language-specific issues into a unified taxonomy. This makes it possible to conduct meaningful comparisons between human-written and AI-generated code across different languages. The dataset employed in this work is The Vault: A Com-

prehensive Multilingual Dataset for Advancing Code Understanding and Generation [19], developed by MIT. This resource contains code snippets in the ten most widely used programming languages. In this thesis, we focus specifically on 280518 C code snippets, which are statically analyzed using clang-tidy V.21.1.0 [32] for defect detection and Semgrep [27] for identifying security vulnerabilities. Thus, the purpose of this work is not only to understand how the problems introduced by the use of LLMs in code development evolve, but also to extend Cotroneo's work [3], where the analysis was conducted on Java and Python using the same proposed methodology. The next chapter provides an overview of related works, while Chapter 3 presents in detail the approach, tools, and datasets employed for this study. Chapters 4 and 5, in turn, present the results obtained and the final considerations, and also offer a perspective on future work.

1.1 Thesis Contribution

The main contribution of this thesis is the implementation of a standardized approach for static analysis of code quality in Large Language Model (LLM)-generated code. This work builds upon and extends the methodology introduced by Cotroneo et al. [3], which was originally applied to Python and Java. In particular, the contribution of this thesis lies in expanding the scope of the study to the C programming language, thus providing a broader cross-language evaluation of de-

fects, vulnerabilities, and structural characteristics in LLM-generated code. We briefly illustrate our workflow to obtain the results shown in the next chapters:

- Dataset preparation: we extracted two subsets (small and medium) of human-written C code from The Vault dataset and filtered them taking in account the number of unique tokens in each docstring. This phase is very important because we wanted to be fair with all the languages (C, Python and Java) and all the LLMs used, but we will describe it better in third chapter. After this first step, we created the AI-generated codes by a prompt engineering process, providing to each LLMs a prompt with the docstring and the signature of the function that will have to be generated. We cleaned the LLMs responses because we were interested in code only and not in all the generated text that could contain a lot of natural language and comments. After this process, the final dataset is composed by five columns (a unique identifier, docstring, human code, DSC code and Qwen code) and 280518 raws (one raw per C code snippet).

- Defects analysis: we used clang-tidy v.21.1.0 to conduct the analysis, doing a 1:1 mapping between each clang-tidy cheker and ODC's Defect Type categories. This was a crucial point to standardize and make the results comparable. At the end of this phase, we found the percentage of clean and defective

functions. Clean functions are those that do not present defects, on the other hand, defective functions are those that present at least one valid defect. For the LLMs we also found a percentage of empty code, due to the prompt engineering and cleaning processes. To overcome this obstacle, we decided to set an acceptability threshold. If (at the end of the cleaning and prompt engineering processes) the number of analyzable files was upper or equal 85% we can consider acceptable all the process.

- Vulnerabilities analysis: we used Semgrep to find the related Common Weakness Enumeration (CWE), classifying them into three severity categories (Error, Warning, Info). Each code snippet could contain more than one vulnerability and, sometimes, the same vulnerability could propagate through different lines. For this reason we used a de-duplication strategy better explained in the third chapter.

- Complexity analysis: we used Tiktoken and Lizard to compute existing complexity metrics as Number Of Line Code (NLOC) and Cyclomatic Complexity Number (CCN). In addiction, we also calculated the function name length and the number of unique token used in the code.

- Comparing results: we have built and commented different charts to show the obtained results in the chapter 4.

Chapter 2

Related Works

The objective of this work is to provide a standard approach and methodology for analyzing code generated by large language models (LLMs) in terms of quality and security. In the literature, several studies exist, but many are not conducted on a large scale and tend to focus either on defect analysis or on vulnerability analysis. Yu et al. [38], for example, investigated the impact of standalone and non-standalone functions using the HumanEval and CoderEval benchmarks. Standalone functions are defined as those that can be developed solely using standard libraries and built-in functions of the language, while non-standalone functions, which represent about 70% of real-world projects, rely on external or contextual information. The analysis revealed that ChatGPT [24] consistently outperformed the other two models considered, CodeGen [23] and PanGu-Coder [28], across tasks in Python and Java, benefiting from its larger parameter

size and stronger reasoning capabilities. Specifically, the models' performance in generating standalone functions was substantially higher than their performance on non-standalone functions. This finding has big practical implications, therefore, non-standalone functions, require the model to correctly consider and utilize external contextual information like variables, APIs, libraries and other types defined outside the function itself. The authors also investigated on how the prompt could impact on the generation effectiveness. They used the original docstring and a new one human-written and found that Python performance was quite stable regardless of which description was used. This suggests that the models learned Python very well during training. Instead for Java, Pangu-Coder [28] worked better with human-written docstring, while ChatGPT and CodeGen worked better with the original docstrings. This show that models trained on one language may be more reliable when faced with different ways of phrasing the same coding task. These results make clear that the formulation of a prompt plays a major role in shaping the performance of LLMs in code generation. Moreover, the models' training procedures themselves can strongly affect the reliability and security of the code they produce, sometimes improving it and sometimes introducing risks.

Improta et al. [13] further explored this dimension by comparing model behavior before and after task-specific fine-tuning. The study began from the public The Stack [15] corpus and constructed a large-

scale code-to-description dataset of Python functions. Functions were extracted from approximately 24 million Python files using the lizard tool [31], and those without docstrings were discarded. They also applied a cleaning pipeline to remove parameter from docstrings and to produce reliable pairs <text-code>. The code was cleaned by removing inline comments, excluding placeholder and test functions, and filtering by length thresholds. Each example was represented as a pair consisting of the cleaned docstring and function signature as input, and the full function body as output. The dataset was split into training, validation, and test sets, yielding a final training set of 4.4 million examples and evaluation sets of 551,641 examples each. Security was checked using Semgrep [27], a tool that can spot issues in code such as security problems, bad practices, and syntax errors. First, the authors used it to filter out poorly written or incorrect functions from the training data, creating a cleaner and more reliable dataset. They then fine-tuned two versions of the same base model: one using all the original data and one using only the cleaned data. Both models were asked to generate the same set of test functions, which were again analyzed with Semgrep to compare their quality. The results showed that specific fine-tuning in general helped the model write better code, and that cleaning the training data made an even bigger difference. The model trained on the cleaned dataset produced code with far fewer issues, without losing accuracy. Overall, the study shows that carefully

selecting and improving training data is a powerful way to improve the quality of code generated by LLMs. Another study focusing on both quality and security of generated code is that of B. Yetistiren et al. [37], which aimed to empirically evaluate the validity, correctness, security, reliability, and maintainability of AI-generated code, providing a comparative basis for both practitioners and tool designers. The experiments employed the HumanEval [22] evaluation set of 164 Python problems, each consisting of a function signature, a descriptive docstring, a canonical solution, and unit tests. The input provided to the tools consisted of the signature and docstring, while the output was the generated code. Evaluation was performed using Python interpreters (for validity), HumanEval unit tests (for correctness), and SonarQube [29] (for static analysis of security, reliability, and maintainability). This study aims to evaluate different versions of state-of-the-art models like Amazon CodeWhisperer [1], GitHub Copilot [8] and ChatGPT [24] in order to assist practitioners and developers in the choose of the best LLM according to their needs. Results shows that ChatGPT and Codewhisperer (Nov. 2022 version) are better than Copilot in code validity and correctness, while, for the security, they identified almost the same score for all the LLMs. Tihanyi et al. [33] examined how secure C code generated by modern LLMs really is. Their goal was to understand whether today's models, even when they produce code that compiles, still introduce serious vulnerabilities. To do this, they cre-

ated a large dataset of AI-generated C programs and used a formal verification tool, Efficient SMT-based Context-Bounded Model Checker (ESBMC) [30] to check each one for common security issues, such as buffer overflows, integer errors, null pointer problems, and memory leaks. The results showed that a large share of the generated programs contained vulnerabilities, and that more complex code tended to have more bugs. So, the authors concluded that automatically generated C code remains highly risky from a security perspective and that strict validation and verification are essential before using such code in real systems. Cotroneo et al. [4] examined the vulnerability of AI code generators to targeted data poisoning attacks, where small portions of training data are replaced with semantically equivalent but insecure code while maintaining unchanged natural language descriptions. The aim was to assess whether small poisoning percentages can compromise model security, whether such attacks are stealthy (do not degrade correctness), and which factors affect their success. To this end, the authors constructed the PoisonPy dataset, yielding 823 intent/snippet pairs (568 safe and 255 unsafe). Vulnerabilities covered 24 CWE categories and they were grouped into three macro-areas: Taint Propagation Issues, Insecure Configuration Issues, and Data Protection Issues. The authors evaluated three LLMs: Seq2Seq, CodeBERT [6], and CodeT5+ [35] and they chose metrics like Attack Success Rate (ASR) and Edit Distance (ED). Results showed that with as little as

3% poisoned data, the pre-trained models produced between 12% and 41% vulnerable snippets, with CodeT5+ reaching the highest ASR. Increasing the poisoning rate to 6% led to ASR values exceeding 80% in some settings. Importantly, they also showed that no statistically significant degradation in code correctness was observed. The analysis highlighted that pre-trained models are both more powerful and more vulnerable to fine-grained poisoning so the authors suggested countermeasures including careful dataset curation, detection techniques such as activation clustering, and mitigations like fine-tuning on trusted datasets or model pruning. Finally, another large-scale study by Cotroneo et al. [3] compared human-written and AI-generated code across Python and Java, analyzing three attributes of software quality: defects, security vulnerabilities, and structural complexity. The dataset extended HMCorp [36] by recovering original docstrings and metadata from [12], resulting in over 507,000 instances of paired human and AI code. Code was generated by ChatGPT [24], DeepSeek-Coder [9], and Qwen-Coder [11] using original docstrings as prompts, followed by extensive cleaning and preprocessing. Defects were analyzed using Pylint [17] for Python and PMD [26] for Java, vulnerabilities with Semgrep mapped to CWE categories, and structural complexity with lizard and tiktoken [10]. To ensure comparability, all rules were mapped into the Orthogonal Defect Classification framework [2]. These results highlight the importance of adopting quality-assurance

workflows specifically designed for AI-generated code, using tools like static analysis and security scanners alongside expert review to reduce potential risks. Going through this line of research, this thesis introduces a unified framework for assessing the quality of human and AI-generated programs considering defects, security issues, and code complexity.

Chapter 3

Methodology

This chapter will show the tools and methodology used to achieve the objective discussed in the previous chapters.

3.1 Dataset

The Vault dataset was chosen for this work, consisting of high-quality text-code pairs (for a total of 10 different programming languages, including Python, Java, and C) extracted from a subset of The Stack dataset. The size of the dataset is as follows:

Table 3.1: Dataset composition adapted from [**?**].

TheVault TOT.	34098775 <text-code> pairs
Small set (C)	105978 <text-code> pairs
Medium set (C)	381207 <text-code> pairs
Final Dataset	280518 <text-code> pairs
Different Repos	78583

The objective of this first phase was to build a human code dataset similar in size to that used in the work of Cotroneo et al. [3] for analysis in Python and Java. This is because the purpose of this work is also to extend the above-mentioned work by performing static analysis of C code snippets. For this reason, the following statistics on docstring length were calculated in order to filter the dataset as fairly as possible:

Table 3.2: Docstring stats (in terms of number of tokens) for The-Vault's small and medium C dataset

Statistic	**Dataset Small C**	**Dataset Medium C**
Total entries	105978	381207
Min docstring length	6	6
Max docstring length	498	498
Docstrings > 15 tokens	61611	218907
Mean length	30.4496	30.3137
Median length	19	19
CDF (Mean)	0.6737	0.6829
CDF (Median)	0.5031	0.5111

From what one can observe for the small dataset, we have projected the average length of the docstring (calculated in terms of number of tokens) onto the cdf, obtaining that about 67% of the dataset possesses a docstring with less than 30 tokens. Not by chance the median turns out to be 19. For the medium dataset the results are not very different. Even in this case the median is 19 and the projection of the mean onto the cdf suggests to us that 68% of the docstrings have less than 30 tokens, while 50% possess at least 19 tokens. To these results was added the desire to respect the constraints of the previous work by

Cotroneo et al. [3], therefore it was chosen to filter the two datasets by extrapolating only the snippets whose docstring had at least 15 tokens. Thus our final human code dataset, is composed of all the entries of the small and medium dataset that have at least 15 tokens in the docstring, for a total of 280518 snippets belonging to 78583 different repositories of real projects.

3.1.1 AI generated Dataset

To build the complete dataset (human code + generated code from LLMs) we performed inference on the following pre-trained models: DeepSeek Coder [9], Qwen [11]. The process of automatic C code generation was executed through the vLLM library [34].

The objective is not to retrain the model (fine-tuning), but to exploit its generative capabilities to produce new code starting from textual descriptions (docstrings) and signatures.

The prompt engineering pipeline applied for the generation of the AI dataset is composed of:

- the vLLM inference engine,
- the sampling parameters (`SamplingParams`).

The vLLM library represents an advanced infrastructure for the execution of large language models, designed to optimize performance in the inference phase, in the generation of new outputs from an already pre-

trained model. In the context of the analyzed code, vLLM acts as the execution engine of the specified model. vLLM receives the prompts built by the script, applies the SamplingParams and manages the text generation, finally returning the C code produced by the model. The `SamplingParams` class defines the rules that determine the model's behaviour during the generation phase. These parameters come into play upon invocation of the `llm.generate()` method.

The parameters set in the code are:

- `temperature=0.0`: this value set to zero serves to eliminate the random component, making the generation as deterministic as possible.
- `top_p=0.9`: applies the nucleus sampling technique, limiting the choice of tokens to the most probable ones, up to reaching a cumulative probability of 90%. This allows balancing coherence and linguistic flexibility.
- `max_tokens=512`: imposes a maximum limit on the length of the generated output, preventing excessively long responses.
- `stop=["<|user|>", "<|endoftext|>"]`: defines the sequences that interrupt the generation, avoiding the inclusion of non-pertinent text.

During execution, vLLM applies these parameters in real time, integrating them into the token-by-token decoding process.

It is fair to observe that, despite these measures, the outputs produced by the LLMs did not always respect the imposed prompt rules. Also for this reason, downstream of the generation, it was necessary to perform a cleaning phase, to obtain in output only the generated C code (eliminating all the comments in natural language inserted by the LLM in question). In this paragraph is shown the prompt engineering procedure through which the structured design of the text message provided to the model was obtained. This process is implemented by the `format_prompt()` function, which builds a complete prompt for each docstring, following the typical schema of instruction-based models adapted from [3].

3.1: formatprompt() function adapted from [3]

```
def format_prompt(docstring: str, signature: str) -> str:


    return f"""
    You are an AI programming assistant, utilizing the
    DeepSeek-Coder model, developed by DeepSeek Company,
    and you only answer questions related to computer
       science.


    ### Instruction:
    C
    '''{docstring}'''
    {signature}
    ### Response:
```

The prompt is articulated in several sections:

- A system block which sets the operational context of the model and the behavior instructions ("You are an AI programming assistant, utilizing the DeepSeek Coder model...").
- The insertion of the docstring, which acts as semantic description of the function to be implemented.
- The C function signature, extracted from the human code, which provides the syntactic structure of reference for the function body.

3.1.2 Cleaning AI Dataset

The codes generated by the LLMs at the end of the prompt engineering process shown in the previous paragraph, were cleaned in such a way as to filter only the code of interest, to maintain an AI dataset of only code (to be precise, C code snippets, with a signature and an implemented body). Referring back to the prompt shown in the previous paragraph, we observe that the produced code must be inserted between two patterns:

- Initial pattern: `'''C`
- Final pattern: `'''`

So at first glance it may seem a very simple operation, to extrapolate from the responses all and only the characters included between the

initial and final patterns. In reality it was not like that, as it was observed that the Large Language Model in question, behaved differently in various responses. Not all of them, in fact, possess the two aforementioned patterns. Another reason is the `max_tokens` parameter of `SamplingParams` (mentioned above). This parameter was set to 512, which implies that once 512 tokens are reached, the LLM's response is blocked. If the code therefore is very long, such as to exceed 512 tokens, it will not be generated completely and thus not only might the opening and closing patterns not be present, but it might also turn out to be syntactically incorrect. The cleaning strategy adopted fundamentally distinguishes 4 cases, further refined internally:

- Case A: the response contains both initial and final pattern
- Case B: the response contains only the initial pattern
- Case C: the response contains only the final pattern and the initial pattern is: `'''`.
- Case D: the response does not contain any pattern.

Case A is certainly the simplest, as one just needs to extract everything included between the initial pattern and the final one. For case B, one starts to extract code from the initial pattern (excluded) and for the closing, an attempt is made by searching for the pattern `'}\n'` which corresponds to the last curly bracket (it should be the function's closing

bracket). In the event that this pattern is not present, attempts are made to search for regular expressions such as:

```
if any(tok in line for tok in
    [';', '{', '}', '#include', 'int ', 'void ', 'return']):
```

Extracting all the lines that possess at least one of these tokens. Also case C is rather simple, as one just needs to change the initial pattern to search for to then fall back again to case A. Case D instead is certainly the most complex, as an attempt is made to extrapolate lines that resemble code based on certain tokens (as in case B, but without an initial pattern). It is important to underline that it is necessary to re-evaluate the cleaning procedure for each LLM and modify it appropriately. Not by chance the algorithm briefly described above, did not turn out to be valid for all the rows of the dataset. It is not guaranteed, in fact, that the LLM understands the instruction entrusted to it. Sometimes in the responses the code was totally absent, while in some cases C++ code was generated instead of C (this happened mostly with Qwen). Numerous cases of hallucination were found (in both LLMs). Despite the clear request of the prompt (docstring + signature) in many responses the LLM generated a complete script that included the function to be developed and other utility functions, including pre-processor directives like `'#include'` or `'#define'`.

In some cases instead it was observed that the LLM (in particular Qwen) developed the body of the function without also reporting its signature. For these reasons, the cleaning was subsequently deepened

in such a way as to:

- Perform a robust check of the opening and closing of the function body (using an active count of the number of `'{'` and `'}'`).
- Perform the check on the presence (or absence) of the function's signature within the generated code, and eventually evaluate the correct point at which to insert it.

It was furthermore fixed a post-cleaning acceptance threshold equal to 15% of the dataset's populousness, in particular, the cleaning process is considered successful if at least 85% of the generated code is analzsable, i.e. it is not missing code and does not possess syntax errors (introduced by the cleaning) that can make the analysis with Semgrep fail.

3.2 Defect Analysis

The goal of this paragraph is to show how we analyzed the dataset to evaluate defects. When we talk about "software quality" we refer to how reliable a software is, in terms of different factors such as: portability, usability, correctness, efficiency, security (and all its attributes), maintainability, reusability. So we can definitely define a "defect" as anything that can threaten the software quality. The tool we choose to perform static defect analysis is clang-tidy v.21.1.0 [32]. In order to obtain a jsonl format at the end of the analysis with clang-tidy,

we used also a clang-tidy-converter tool [39]. We also need a strategy to make our results comparable between different languages. For this reason, each clang-tidy checker was mapped into one of ODC's Defect Type categories. The following subsection will show details about the complete workflow and tools of defect analysis.

3.2.1 Clang-tidy v.21.1.0

Clang-Tidy provides a framework dedicated to static analysis, managing the parsing of ASTs (Abstract Syntax Trees) and the generation of diagnostics. Several functionalities are integrated into the core of Clang-Tidy, such as the detection of the compilation environment, the automatic application of corrections and the relative management of conflicts, in addition to the structured presentation of the results. The checks of Clang-Tidy are categorized in groups, defined based on a broad concept—e.g. performance problems, modernization or constructs prone to errors (bugprone)—or based on specific guidelines, such as the C++ Core Guidelines. Among these categories, for the purposes of the analysis, we decided to eliminate `abseil-` and `objc-` . Furthermore we also excluded all warnings of type `clang-diagnostic-` since referred to compilation errors due to the fact that the codes subjected to the analysis were simple C snippets, without declarations of types defined by the programmers, nor of libraries and/or header files (they were extrapolated from real world

projects). Given that each programming language has its own peculiarities, it is fair to observe that the choice of static analysis tool is based on a specific language. In fact it is logical that, changing language, the tool for static analysis used will also change, and , consequently, the checks performed will be significantly different. For this reason it is necessary to standardize the results, as we would like to be able to compare them with those obtained from other languages (Cotroneo et al. [3]). To this end, it was decided to map individually each `clang-tidy` check into a specific defect-type category of the Orthogonal Defect Classification (ODC) by Chillarege et al. [2] .

3.2.2 Orthogonal Defect Classification (ODC)

The ODC is a measurement and categorization technique that extracts semantic information from software defects through classification into a specific defect type. The orthogonality of the defect types, ensures that each of them is in a mutually exclusive relationship with the others.

The two key attributes of the ODC are:

- Defect Type: indicates the nature of the correction to apply to resolve the defect
- Trigger: the event that triggered the activation and propagation of the defect.

This methodology was created to provide rapid and effective feedback to software development teams (in our case) in contrast to traditional quantitative techniques that often limit themselves to counting defects (bean counting) or rely on complex qualitative analyses (such as Root cause Analysis). Thanks to the Orthogonal Defect Classification, quantitative methods can be complemented by the semantics of the defects, to identify more accurately the critical points of the software and the trend in the distribution of defects. Applying ODC for example, not always is a positive trend of defects strictly linked to poor code quality, but it could be an effect limited to a specific category of defect type.

These types include:

Function: defects that influence functionalities aimed at the end user, APIs, global structures and all those that would require a formal design modification.

Assignment: defects that concern the initialization of variables or data structures.

Interface: defects in the interaction with other components, modules or device drivers, via macros, calls and parameter lists.

Checking: logic defects, errors in the conditions of loops or ifs, etc...

Timing/Serialization: defects in the management of shared resources and in real time operations.

Build/Package/Merge: defects that occur due to errors in library systems (import) or in version control.

Documentation: errors that impact the documentation associated with the software both in terms of publications (public documentation intended for end users) and in terms of maintenance (internal documentation used by development teams to understand the software and facilitate future interventions).

Algorithm : efficiency or correctness problems that influence a functionality and can be resolved by applying an algorithm or a data structure without modifying the design.

This shows a mapping example between clang-tidy checkers and ODC Defect Type's categories:

Table 3.3: Mapping example of Clang-tidy checkers into ODC Defect Type

Defect Type	**Clang-tidy Check**
Algorithm/Method	`performance-inefficient-algorithm`
Build/Package/Merge	`modernize-deprecated-headers`
Timing/Serialization	`concurrency-mt-unsafe`
Function/Class/Object	`google-explicit-constructor`
Interface	`cppcoreguidelines-no-malloc`
Documentation	`readability-magic-numbers`
Checking	`hicpp-multiway-paths-covered`
Assignment	`modernize-use-auto`

For example, the checker 'modernize-use-auto' is responsible for the utilization of type 'auto' for the variables declarations. In particular,

it ensures that a specific value is not assigned to an auto variable if the type of that value is known a priori. This assure an improvement also in code readability and mantainability:

```
int value = 13;
HouseStruct &H = Key.getInfo();

// Should not become:

auto value = 13;
auto &H = Key.getInfo();
```

The semantics of this checker, suggests that the correct category of defect type in which to map it is Assignment. The process of mapping was carried out by multiple people in a collaborative manner where discrepancies were present at the level of interpretation of the various defects identified by each individual checker.

3.3 Vulnerabilities Analysis

Software vulnerabilities arise when programs contain flaws that attackers can exploit, potentially compromising security goals like keeping data private, maintaining integrity, or ensuring availability. Information about widely known weaknesses is maintained by Mitre in the Common Weakness Enumeration (CWE) database [21], which also links each weakness to corresponding entries in the Common Vulnerabilities and Exposures (CVE) system. CWE organizes these issues from different perspectives, including software design, develop-

ment practices, and hardware considerations. Following the approach proposed in this thesis, we conducted a vulnerability analysis on the dataset by applying Semgrep to all C code snippets.

3.3.1 Semgrep

For the analysis of vulnerabilities we used Semgrep [27], a tool that understands the textual structure via a semantic analysis of code's schemas. `Semgrep` works by identifying certain rules within a code. It does not exclude the possibility for the user to define their own rules in `YAML` format. As regards the analysis carried out on the dataset, a first script takes care of partitioning the complete dataset into batches each of size equal to 20000. This is because, every time `Semgrep` is launched on a snippet, it is necessary to perform the login, download the rules indicated in the command and execute the analysis. It is obvious that repeating all these operations for each of the snippets is too onerous, the execution times would grow exponentially. Therefore it was chosen to launch `Semgrep` on one batch at a time, where each batch is composed of 20000 snippets. Given that the original dataset is composed of 280518 snippets of code, `Semgrep` will be launched (from scratch) only 15 times (15 batches each of 20000 code snippets). The output will be 15 files in `jsonl` format that contain the results of the analysis with `Semgrep` and will be used later to calculate the statistics of our interest. A second script will take care of calculating

certain statistics on the files obtained in output from the previous code execution. This calculates metrics that allow identifying three types of outcomes:

- **Functions that present errors:** all and only the functions for which `Semgrep` failed the analysis (probably due to the presence of syntax errors).
- **Defective Functions:** number of unique files that contain at least one security vulnerability and have not generated errors. To this end the deduplication technique is used to count as a single instance every function that presents at least one vulnerability found by `Semgrep`, following the removal of files with errors.
- **Clean Functions:** number of functions that were correctly scanned and that do not present errors nor vulnerabilities.

In addition to this main subdivision, it calculates other detailed statistics:

- **Total Security Problems:** counts the number of unique vulnerabilities found (following deduplication). One must keep in mind in fact that a single code snippet can generate more than one vulnerability.
- **Top 10 CWE:** to count the occurrences of each CWE and extrapolates the top 10 from them.

- **Unique CWE Types:** counts the number of unique CWEs identified in the entire dataset.
- **Counts by Severity:** grouping of CWE counts based on their severity (Error, Warning or Info).

The deduplication fundamentally is present in three different ways:

1. **Deduplication at file level (for Errors and Results)**

   It is performed to count how many unique files have errors or results, through the use of a set (which by definition cannot contain duplicates). Within this set are inserted the identifiers of a snippet that generated an error in `Semgrep`. If this identifier is already present in the set, then it is ignored.

2. **Deduplication (and normalization) at label level (for CWEs)**

   Not always in the results obtained from the analysis with `Semgrep`, is only the pattern: `"CWE-XXX"` present, rather it often happens to also get an additional explanatory string of the CWE in question in the "metadata" field. For example: `"CWE-123"` and `"CWE-123: Buffer Overflow"`. For this reason, the aim is to ensure that `"CWE-123"` and `"CWE-123: Buffer Overflow"` are not counted as two different CWEs, but as a single one. A dictionary is therefore used which, for each CWE

number, maps the description (if present). In the end it is chosen to apply the longest and most complete description among all those with the same number.

Example:

```
CWE-123:  "", "XXX", "Buffer Overflow"
output:
CWE-123:  Buffer Overflow.
```

3. **Deduplication at problem level**

   Serves to count how many unique vulnerability reports exist. Sometimes, `Semgrep` can report the exact same problem multiple times (perhaps from rules different ones applied to the same snippet). An identifier is created (an `issue_id`) for each problem, composed of:

   - The path (identifier of the single snippet)
   - The cwe, standardized as seen previously
   - The severity (Error, Warning, Info)
   - The lines (the incriminated code).

   The errors are collected in an initial list (`total_errors`). Since in the same file or in the same function an error can appear more than once, a set (`dedup_err`) is built containing only the identifiers of the functions that produced at least one error.

3.4 Complexity Analysis

To evaluate the complexity of human and AI generated codes, two tools in particular were used: `lizard` [31] e `Tiktoken` [10]. Before presenting them it is right to list the complexity metrics that it was decided to calculate for the purpose of this work:

NLOC (Number of Non-Comment Lines of Code): quantifies the number of lines in a function that contain executable source code. `Lizard` calculates it excluding the lines that are completely empty or that contain exclusively comments, starting from the base value 1.

CCN (Cyclomatic Complexity Number): software metric that measures the logical complexity of a function. `Lizard` calculates it starting from a base value of 1 (which represents a single execution path) and adds +1 for each structure that introduces a branch in the control flow (e.g. `if`, `case`, `while`, `for` etc.).

Token_count: number of unique tokens used within a single code fragment.

Function_name_length: length in terms of characters of the function name, excluding its type and parameters.

Unique_token: total number of unique tokens used by an Author (human, DSC, Qwen).

Lizard

`Lizard` is a static code analyzer, which implements processors for the calculation of metrics like `NLOC`, `CCN` etc. Furthermore it uses specific readers (based on the extension of the file to be analyzed) capable of performing the parsing of the code and the generation of tokens.

Table 3.4: Lizard processors and their function.

Processor	**Description**
`preprocessing`	Normalizes and cleans tokens
`comment_counter`	Counts comment lines
`line_counter`	Counts significant lines (NLOC)
`token_counter`	Counts significant tokens
`condition_counter`	Increments CCN (if, for, etc.)

A reader recognizes where a function begins and ends, keeps track of indentation and of entry/exit from `'{'` blocks for example. Every time `Lizard` analyzes a file, it creates an instance of the `FunctionInfo` class to save all the information concerning the analyzed function. Informations like:

- `cyclomatic_complexity` (CCN): The cyclomatic complexity number.
- `nloc`: The number of lines of code (NLOC) of the function.
- `name`: The short name of the function (without return type and parameters).

- `long_name`: The long name, which includes parameters and return type (e.g. `void my_function(int a)`).
- `start_line` / `end_line`: The start and end lines of the function in the file.
- `filename`: The name of the file it belongs to.

In the code used for this analysis the function analyze_source_code(filename, code) is called, which corresponding to a specific filename extension, takes care of creating a context and selecting a suitable reader. This reader will take care of generating the tokens by performing the parsing of the code and returning an iterator from it. For each token all the processors are called and, based on the specific logic, the metrics are updated. The function returns an instance of the `FileInformation` class, which contains the context and the calculated metrics on the specific file. Furthermore it possesses an additional field, `function_list` which allows us to iterate through all the functions that were found by lizard within the filename.

Tiktoken

Large language models do not see natural text as we humans see it, but rather in the form of sequences of numbers (tokens). An algorithm to convert text into numerical sequences is Byte pair encoding (BPE).

`Tiktoken` is a tokeniser that uses `BPE` to convert text into tokens, exploiting the power of OpenAI models (`GPT-4` [25]). The process of obtaining the correct encoder (encoder) is a procedure in multiple stages that abstracts the underlying complexity. The API used in the code is `encoding_for_model(model_name)`. It is a factory function of a high level that acts as an intermediary: its purpose is to map a readable model name (e.g., `"gpt-4"`) to the technical name of the specific encoding used by that model (e.g., `"cl100k_base"`). To achieve this result, the function first consults a series of dictionaries with direct mapping, `MODEL_TO_ENCODING`, which list the exact matches. Once the encoding name is resolved, `encoding_for_model` internally invokes the lower-level function `get_encoding(encoding_name)`. The latter is the one truly responsible for the assembly of the encoder object. The main component is the vocabulary, technically defined in the code as `mergeable_ranks`. This is not a simple list of words, but rather the complete "merge" table resulting from the training process of Byte Pair Encoding (BPE). Physically, it is a resource file that defines the complete mapping between the byte sequences (the elementary tokens and the merged ones) and their corresponding integer identifiers (IDs). For our purpose the `gpt-4` encoding is used. In the first phase, the code iterates over each record of the dataset and, for each predefined code category (like `human_code`, `qwen_code`, etc.), performs a detailed analysis. Using the `lizard` library, the

function-level metrics (`NLOC`, `Cyclomatic Complexity`) are extracted and, via `tiktoken`, the number of tokens specific to the `"gpt-4"` model is calculated. This data is then aggregated into distinct containers for each field: a list (`defaultdict(list)`) accumulates the metrics of all the functions belonging to that field, while a set (`defaultdict(set)`) collects the vocabulary of unique tokens for the same. The second phase consists of the statistical processing of this data. Initially, the script calculates and presents the descriptive statistics (mean,min, max, standard deviation) distinctly for each field, allowing a direct comparison between the different categories. Subsequently, it performs an aggregated analysis: all the function metrics, regardless of the field of origin, are unified into a single "flat" collection (`all_metrics`) and all unique tokens are merged into a single global set (`all_tokens`). On these aggregated collections the statistics are recalculated, thus providing an overall view.

Chapter 4

Experimental Results

In this chapter, the results of the experiments on the AI-generated dataset, built as seen in the previous chapter, will be shown.

4.0.1 Defects Assessment

The clang-tidy v. 21.1.0 checkers were executed on the AI-generated dataset (and also on the human code) and gave the following results:

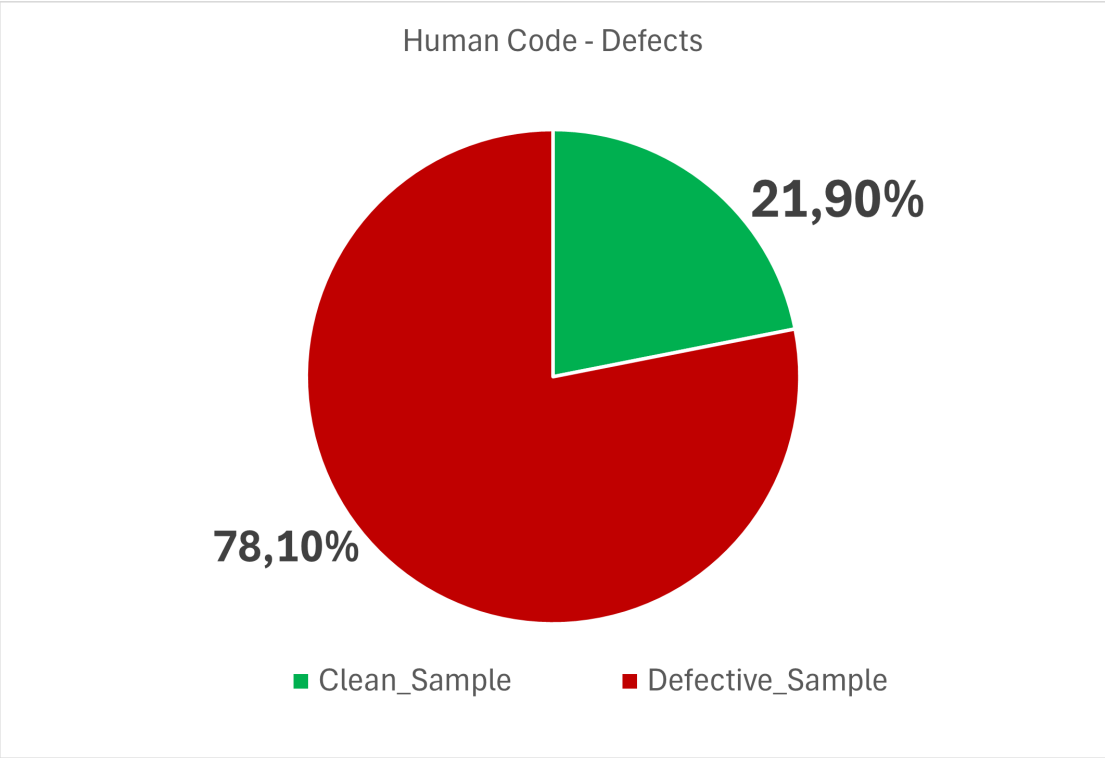

Figure 4.1: Percentage of defective and clean function - Human

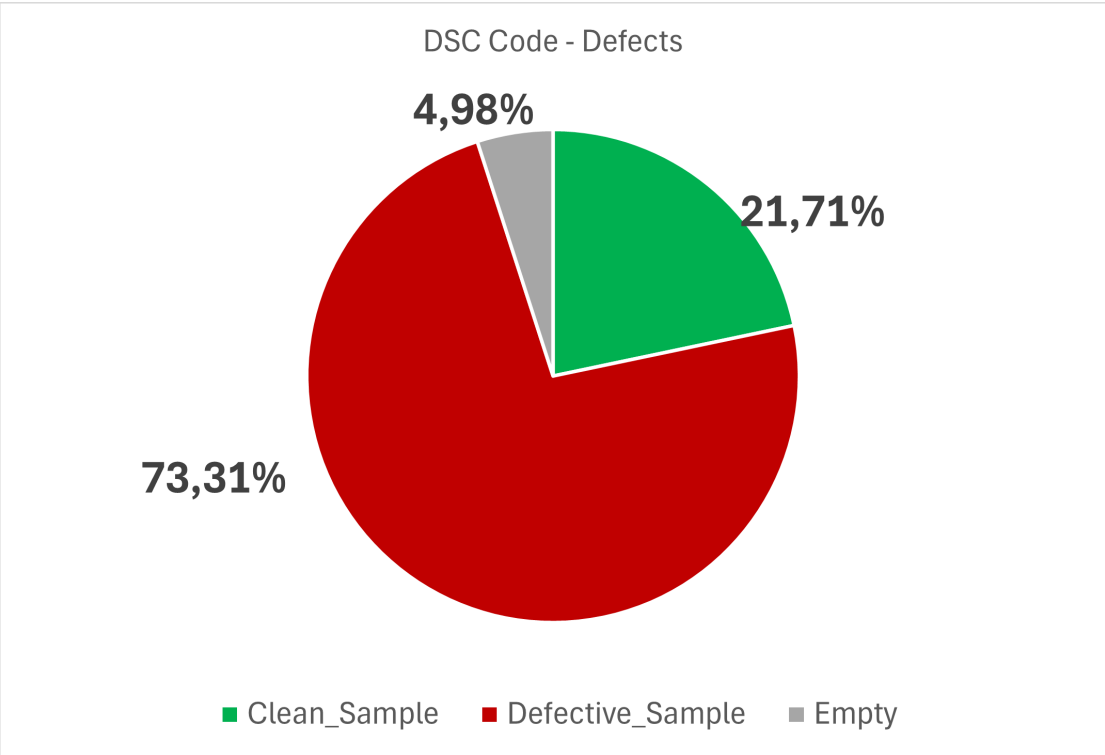

Figure 4.2: Percentage of defective and clean function - DSC

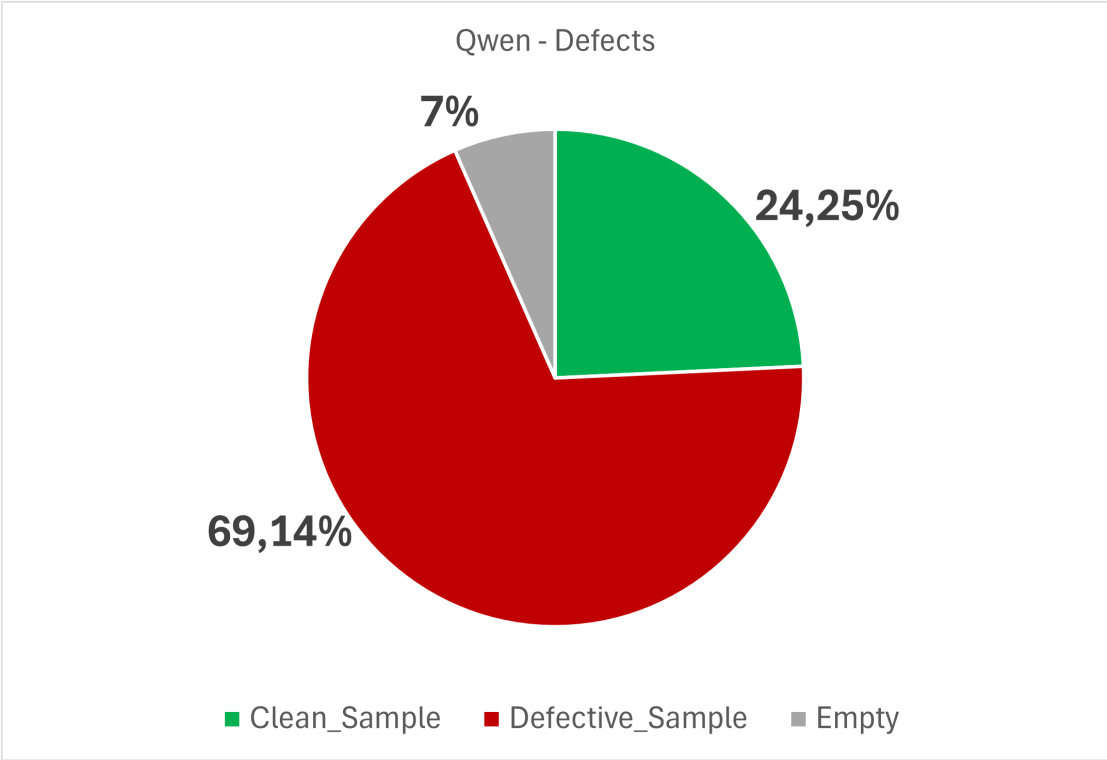

Figure 4.3: Percentage of defective and clean function - Qwen

Human code presents the highest defects percentage, while Qwen seems to be the less defective. The following bar chart represents the aggregated statistics:

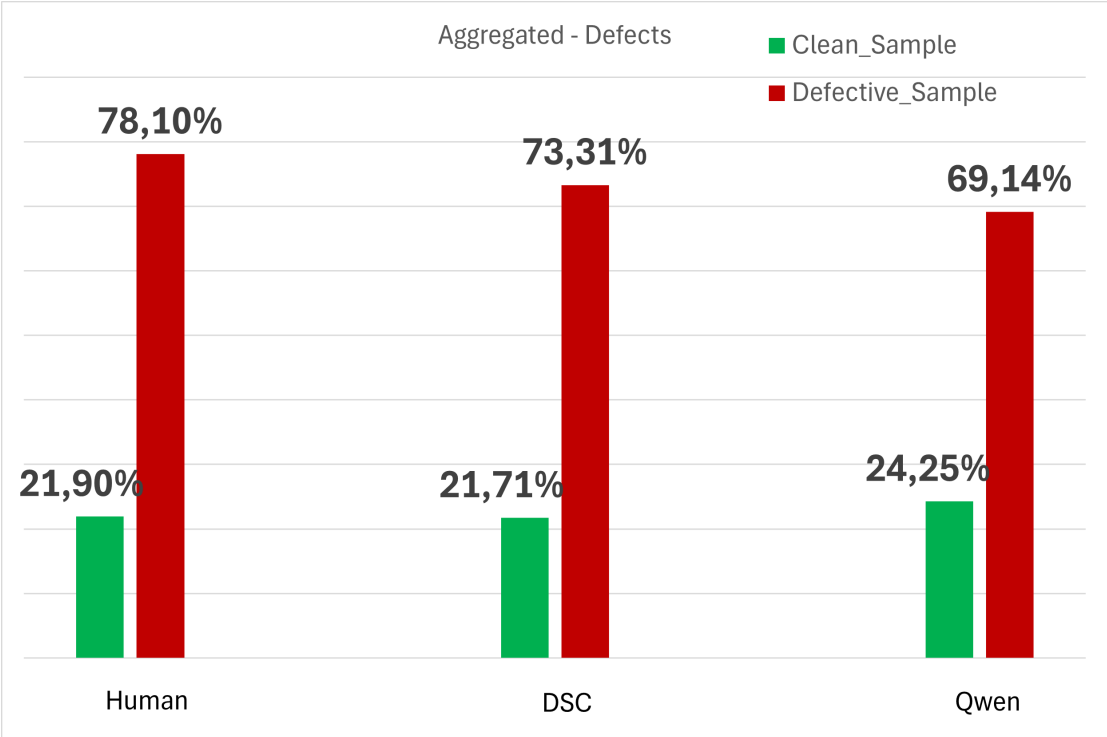

Figure 4.4: Aggregated view of clean and defective functions by Author

To better understand those outcomes, it must be remembered that when AI code was generated, not all the lines of qwen code and dsc code were populated. This was due to the problems discussed in the third chapter, but also to the cleaning process applied on the dataset. For this reason, the percentage of 'empty code' is present only in Qwen and DSC's pie charts and this must be kept in mind because it could influence the outcomes. In fact, it seems that human code has more defects than DSC and Qwen (but if, for example, the percentage of clean functions obtained without cleaning qwen code is considered, it is about 0.99%). Another aspect to take into account is the type of the defects, so it can be understood if human defects are more dangerous than LLM's defects.

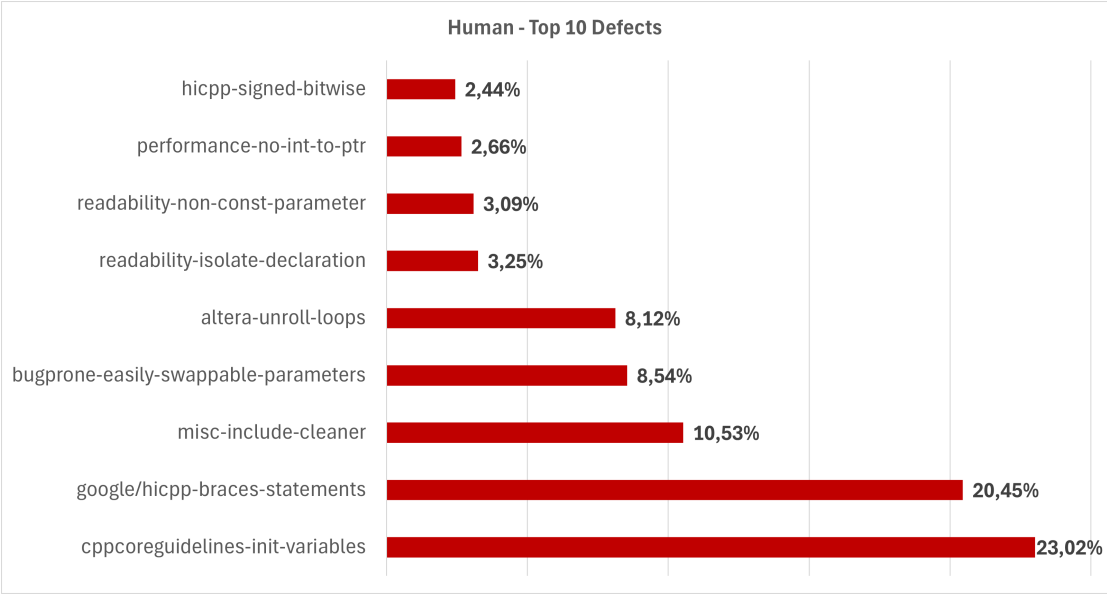

Figure 4.5: Percentage of top 10 defects - human

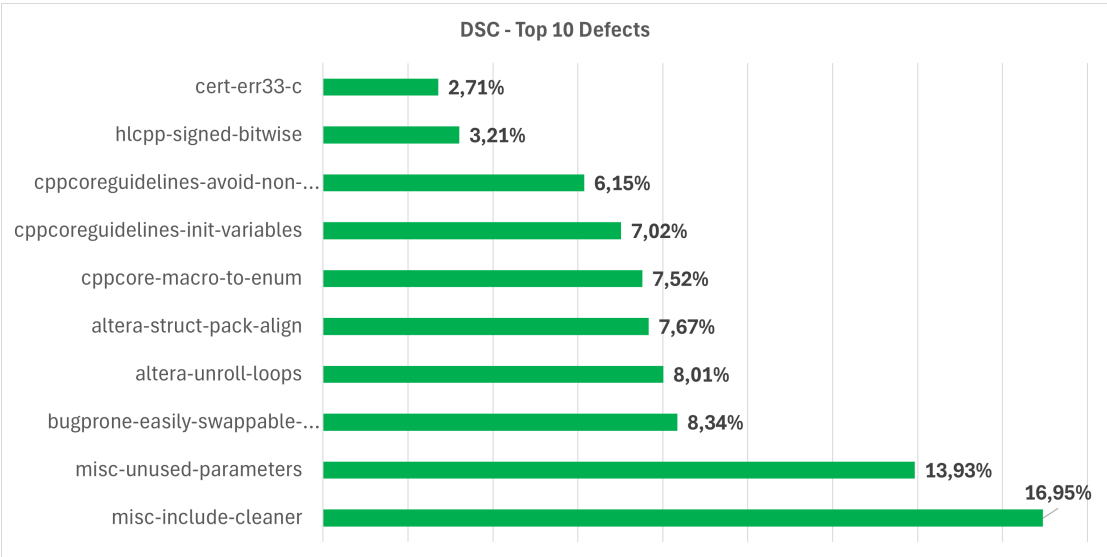

Figure 4.6: Percentage of top 10 defects - DSC

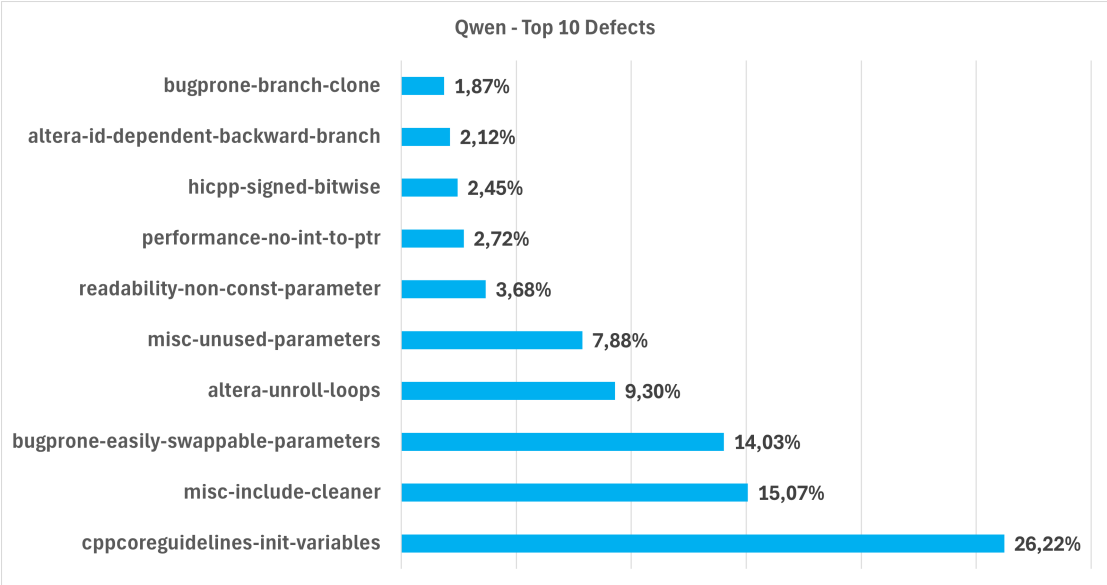

Figure 4.7: Percentage of top 10 defects - Qwen

Human code presents 23,02% cppcoreguidelines-init-variables (Assignment) defect type, immediately followed by readability-braces-around-statements (20,45%) and misc-include-cleaner (10,53%). DSC code presents 16,95% misc-include-cleaner (Function/Class/Object) defect type, followed by misc-unused-parameters (13,93%) and bugprone-easily-swappable-parameters (8,34%). Qwen code presents 26,22% cppcoreguidelines-init-variables defect type, followed by misc-include-cleaner (15,07%) and bugprone-easily-swappable-parameters (14,03%). Here is an explanation of the top 3 defect categories of each source (human, dsc and qwen):

- `Cppcoreguidelines-init-variables`: it serves to find local variables that are declared without being initialized (that is, without an initial value). The goal is to prevent a very common and dangerous error: using a variable before it has been assigned

a value. This leads to undefined behavior or produces random and unpredictable results.

- `Readability-braces-around-statements`: Checks that the body of an if or a loop happens inside the braces.

- `Misc-include-cleaner`: it performs the check of missing or unused includes. It is important to underline that in point of view of human-code, it does not have much value as the extracted functions are composed only of code snippets and therefore does not possess pre-compilation directives such as `#include` or `#define` or `#ifdef`. It is much more interesting to observe the behaviour of this checker in the codes generated by LLMs as the latter contain the directives mentioned above.

- `Bugprone-easily-swappable-parameters`: It performs the check on the parameters of a function and triggers the warning where two parameters of the same type are placed one next to the other.

- `Misc-unused-parameters` : one or more input/output parameters of the function are not really utilized in it.

- `Altera-unroll-loops`: Missed optimizations of loops in Intel FPGA hardware.

Before discussing about the results we have to take a look at the defect

type classification:

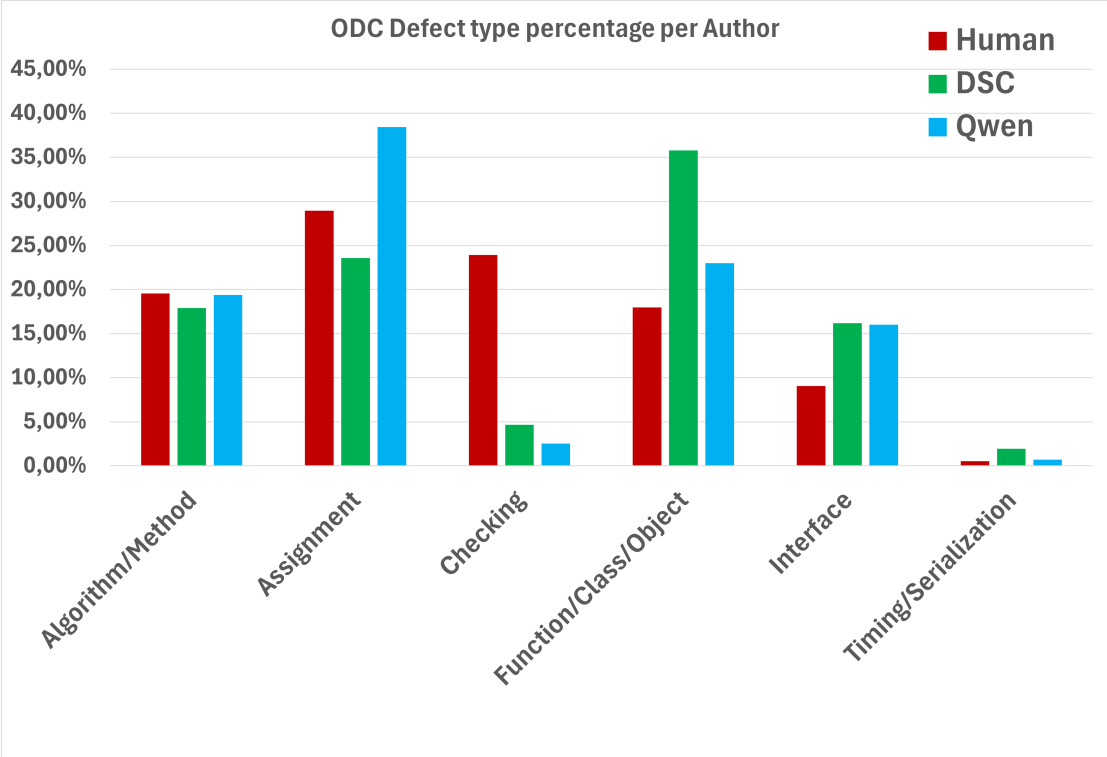

Figure 4.8: Percentage of defects per ODC categories - Aggregated results

Regarding the nature of the defects, human-written code was dominated by the Assignment category (291,231 defects), primarily caused by cppcoreguidelines-init-variables (uninitialized variables), which alone accounted for 231,642 instances. The second most problematic category was Checking (240,667), largely due to readability-braces-around-statements (missing braces). Conversely, DSC-generated code was dominated by Function/Class/Object defects (224,900), with misc-include-cleaner (106,578) being the most common issue. Its second-largest category was Assignment (148,221), where the principal fault was misc-unused-parameters (87,606). Qwen's code, similar to the

human baseline, was most affected by Assignment defects (258,199), again driven by cppcoreguidelines-init-variables (176,241). Its second-most frequent category was Function/Class/Object (154,447), also led by misc-include-cleaner. The complete distribution of ODC defect types is detailed in Table 4.1.

Table 4.1: Distribution of ODC Defect Types

ODC Defect Type	**Human**	**DSC**	**Qwen**
Assignment	291,231	148,221	258,199
Function/Class/Object	180,703	224,900	154,447
Algorithm/Method	196,776	112,472	130,279
Interface	91,336	101,742	107,453
Checking	240,667	29,256	17,012
Timing/Serialization	5,569	12,211	4,706
Total	**1,006,282**	**628,802**	**672,096**

In addition, if only the defective functions are considered, it can be seen that human code presents the highest average of defects per sample:

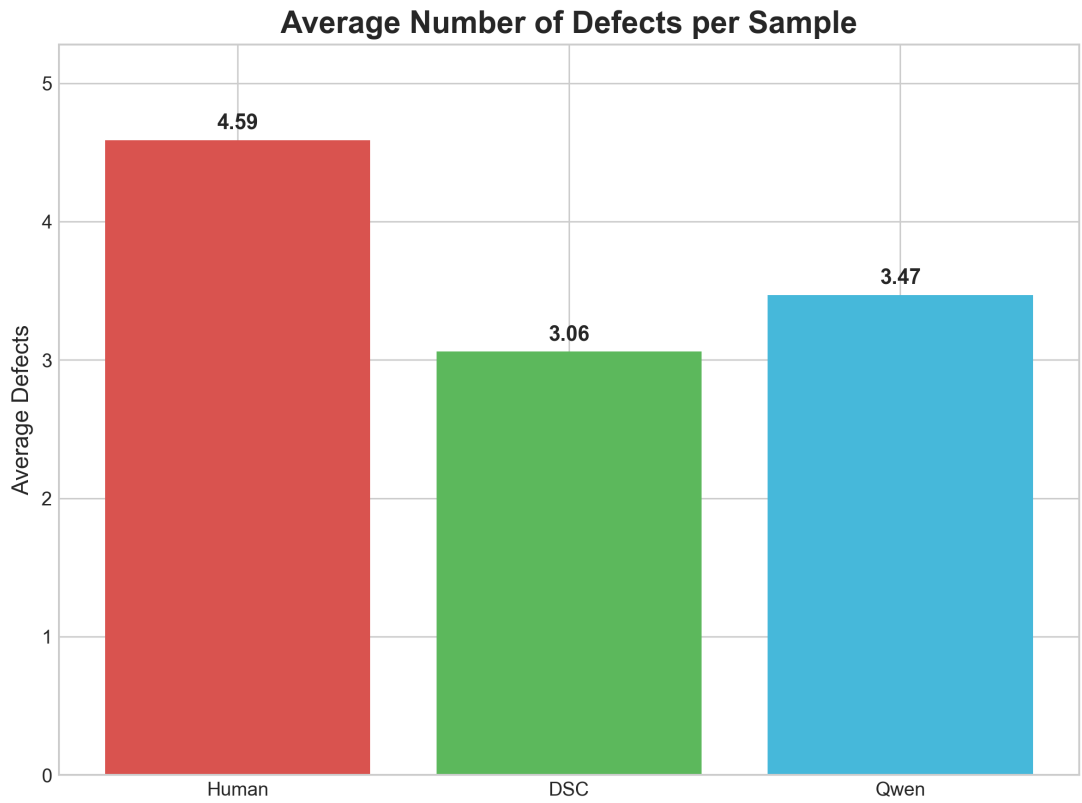

Figure 4.9: Average defects per sample

Human-written code exhibited the highest total number of defects (1,006,282) and the greatest density of defects per sample (4.59). It presented a 78.10% rate of defective samples (219,078) while, to take in account also syntax errors it will be more appropriate reading Semgrep's results. The DSC model registered 628,802 total defects, with an average of 3.06 defects per sample; 77.16% of its analyzable samples were found to be defective (73.31% considering also the empty lines in the Dataset, as seen before), though it must be noted that 13,972 files were empty and thus not analyzed. The Qwen model presented 672,096 total defects, averaging 3.47 per sample. It had the lowest rate of defective samples (74.04% normalizing by analyzable files and

69.14% normalizing by dataset's total entries number) but the highest number of empty, unanalyzed files (18,541). The code used to perform the analysis calculates statistics normalizing on the number of total analyzable files (that is not the same for human, DSC and Qwen code). All the pie charts shown before, are normalized on the number of dataset's total entries (280518 C code snippet, that is a constant value). If we consider the obtained results normalized by the number of analyzable file per Author (human, Qwen and DSC), a summary of these findings is presented in Table 4.2. It is still possible to admit that human code presents the higher percentage of defects, while Qwen results to generate the less defective code but at the same time the major number of empty C snippet.

Table 4.2: Summary of Code Defect Statistics

Metric	**Human**	**DSC**	**Qwen**
Total Defects	1,006,282	628,802	672,096
Defective Samples	219,078	205,659	193,964
Defective Samples (%)	78.10%	77.16%	74.04%
Empty / Non-Analyzed	0	13,972	18,541
Avg. Defects per Sample	4.59	3.06	3.47

4.0.2 Vulnerabilities Assessment

This analysis compared security vulnerabilities using Semgrep, with findings mapped to the Common Weakness Enumeration (CWE) classification.

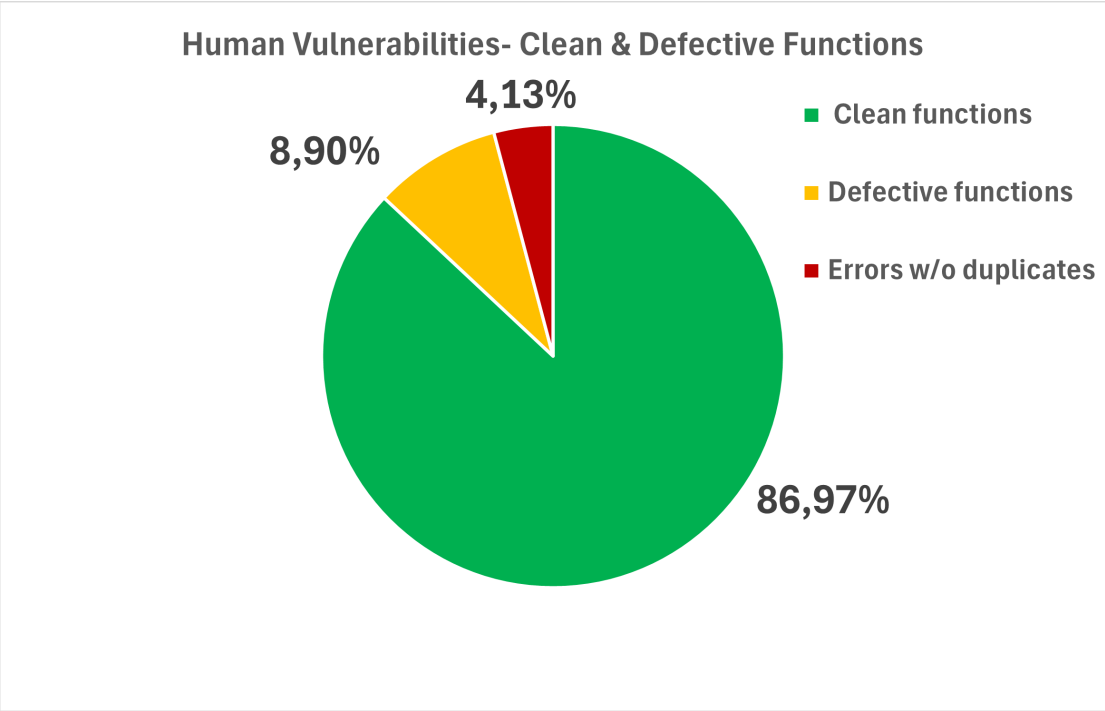

Figure 4.10: Human Vulnerabilities - percentage of clean and defective function

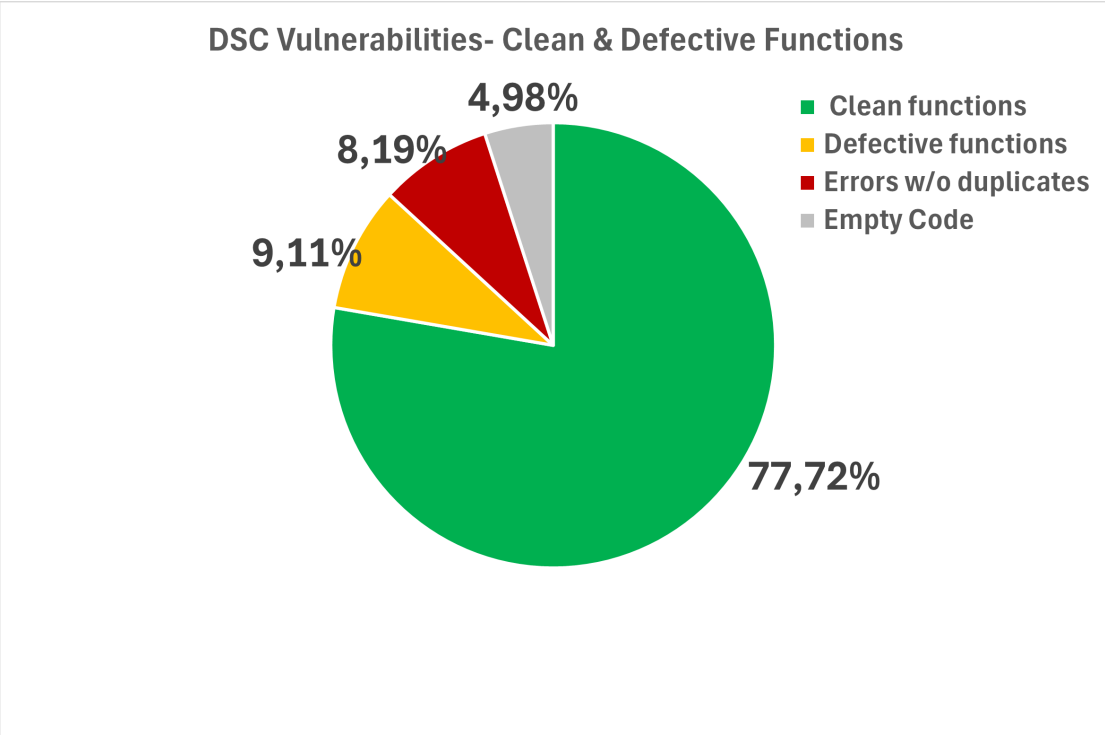

Figure 4.11: DSC Vulnerabilities - percentage of clean and defective function

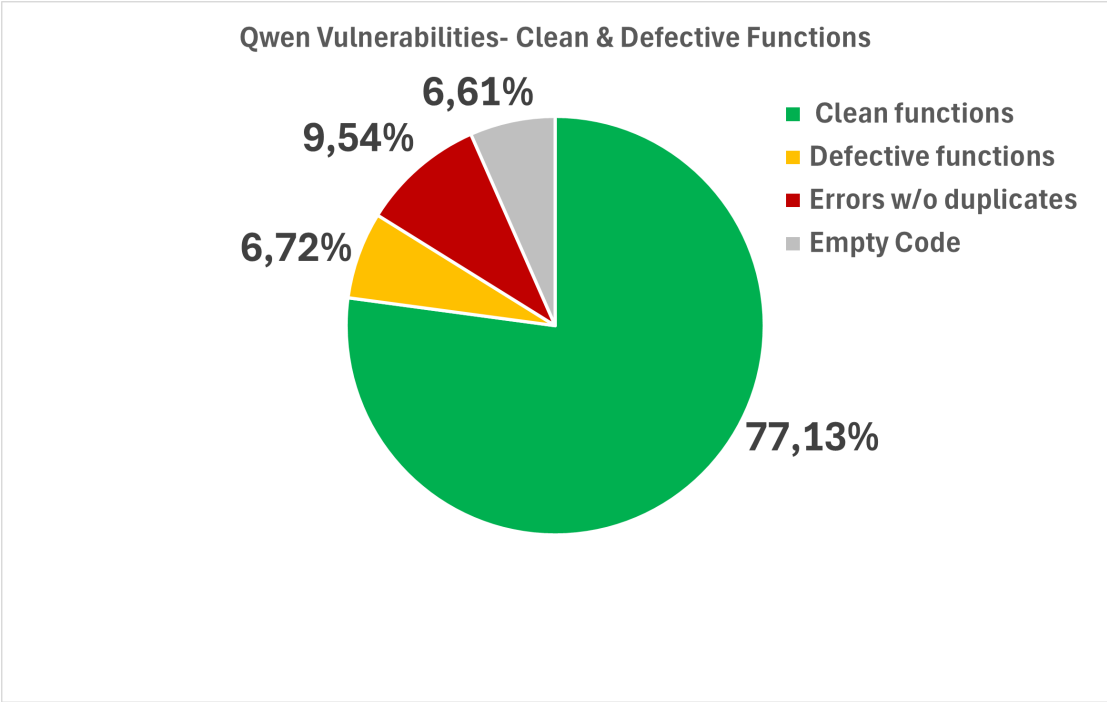

Figure 4.12: Qwen Vulnerabilities - percentage of clean and defective function

Human-written code registered the highest number of total security issues (58,061), with 8.90% (24,976) of functions identified as defective and 86.97% as clean. The rate of skipped or erroneous files was 4.13%. The DSC model recorded 49,907 total issues, with a slightly higher rate of defective functions at 9.11% (25,551) and 77.72% clean functions. However, it presented a significantly higher rate of non-analyzable files (6.72% errors plus 13,972 empty files). The Qwen model demonstrated the strongest performance in security, registering the lowest number of total issues (36,973) and the lowest percentage of defective functions (6.72%). This result is balanced by the highest rate of non-analyzable files, combining 9.54% errors with 18,541 empty files. A summary of these vulnerability statistics is provided in Table 4.3.

Table 4.3: Summary of Security Vulnerability Statistics by Author (considering empty code as clean functions)

Metric	**Human**	**DSC**	**Qwen**
Total Scanned Functions	280,518	280,518	280,518
Total Issues	58,061	49,907	36,973
Defective Functions	24,976	25,551	18,839
Defective Functions (%)	8.90%	9.11%	6.72%
Clean Functions (%)	86.97%	82.70%	83.75%
Errors / Skipped (%)	4.13%	8.19%	9.54%
Unique CWEs Found	28	29	27

Now let us take a look at the top 3 CWE in every outcome:

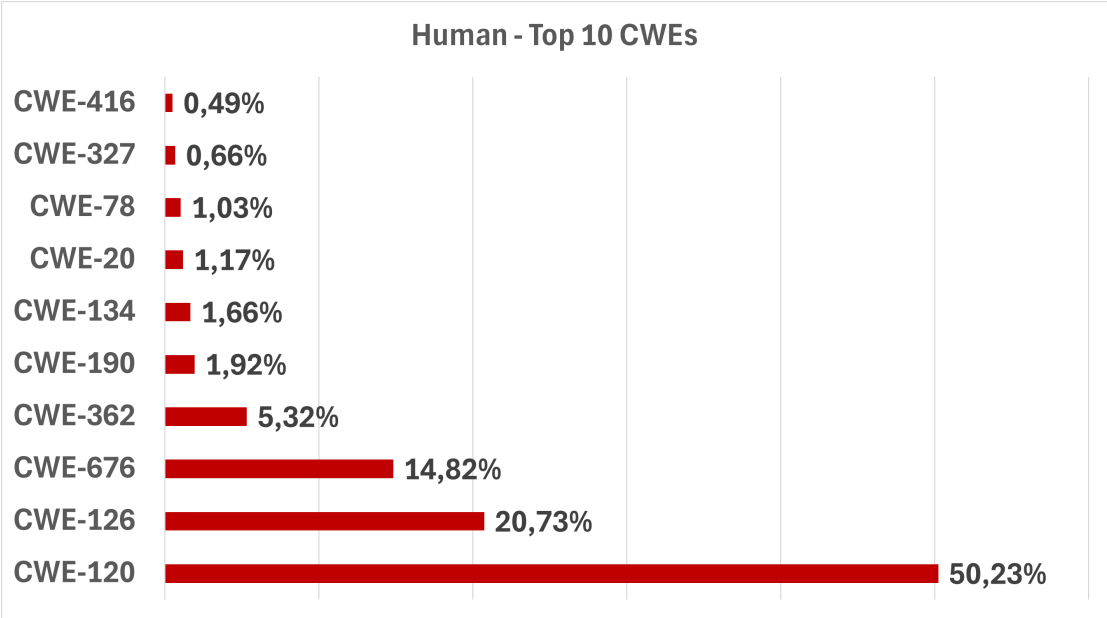

Figure 4.13: Human - Top 10 Vulnerabilities

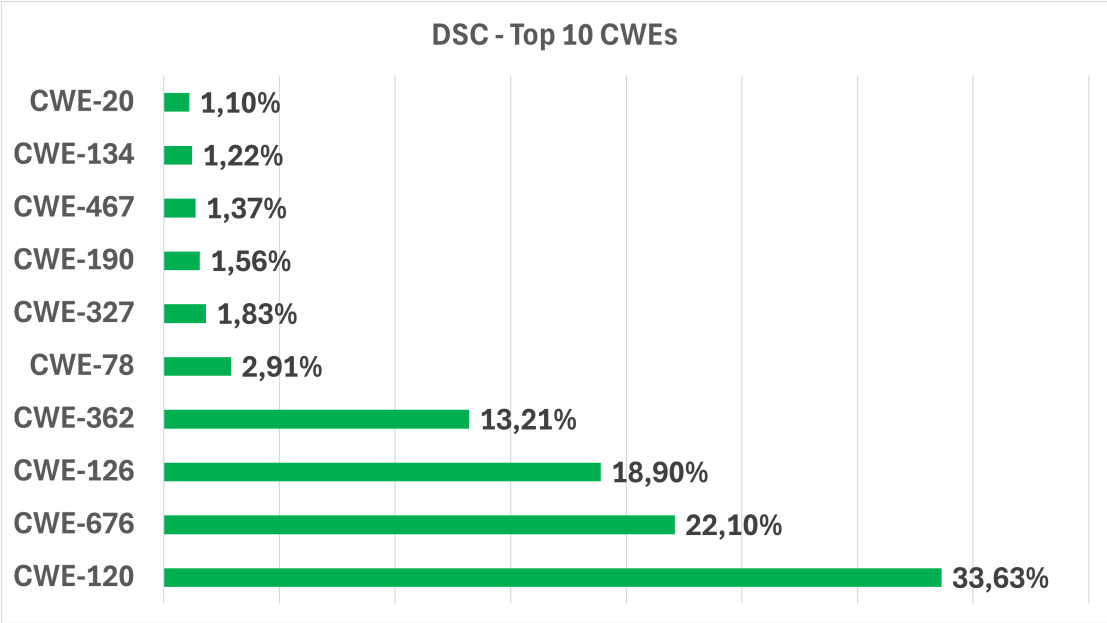

Figure 4.14: DSC - Top 10 Vulnerabilities

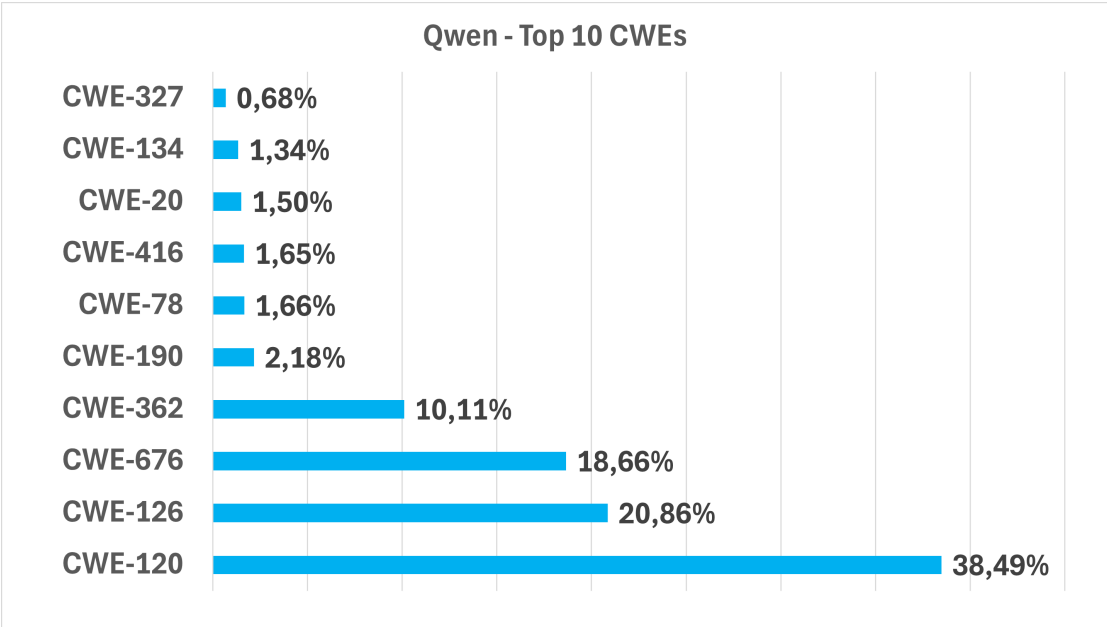

Figure 4.15: Qwen - Top 10 Vulnerabilities

- `CWE-120 (Buffer overflow)`: during a data transfer it is not verified if the dimension of the input fits into that of the destination.

- `CWE-126 (Buffer over-read)`: The buffer is read outside of its imposed limits. Therefore it flows into an unknown memory area.

- `CWE-676 (Use of potentially dangerous function)`: Identifies the use of function calls that are known to be difficult to use safely and that, if used improperly (even involuntarily), almost certainly introduce other serious vulnerabilities.

A significant finding was the commonality of dominant vulnerabilities across all authors. CWE-120 (Buffer overflow) was, by a substantial margin, the most frequent vulnerability for all three sources. For human code, the top three issues were CWE-120 (29,164 instances), CWE-126 (Buffer over-read, 12,035 instances), and CWE-676 (Use of potentially dangerous function, 8,606 instances). The AI models followed an identical ranking for their top three vulnerabilities, differing only in magnitude. DSC's code contained 16,786 instances of CWE-120, 11,027 of CWE-676, and 9,431 of CWE-126. Qwen's code showed 14,231 instances of CWE-120, 7,713 of CWE-126, and 6,900 of CWE-676. The variety of unique vulnerabilities was almost identical (27-29). Table 4.4 illustrates the frequency of the top vulnerabilities across sources.

Table 4.4: Frequency of Top 5 CWEs by Author

CWE ID	Weakness Name	Human	DSC	Qwen
CWE-120	Buffer Overflow	29,164	16,786	14,231
CWE-126	Buffer Over-read	12,035	9,431	7,713
CWE-676	Use of Dangerous Function	8,606	11,027	6,900
CWE-362	Race Condition	3,089	6,593	3,739
CWE-190	Integer Overflow	1,116	777	807

4.0.3 Complexity Assessment

This analysis investigated structural complexity using metrics for Number of Lines of Code (NLOC), Cyclomatic Complexity (CCN), Token Count (TC), Function Name Length (FNL), and Unique Tokens (UT). The results show that human-written code is drastically more complex, longer, and more logically intricate than the AI-generated code. Human code also demonstrated a significantly richer programming "vocabulary", employing 60,120 unique tokens. This was substantially higher than both AI models, which registered approximately 50,000 unique tokens (50,195 for DSC and 50,360 for Qwen), indicating that the AI models tend to reuse a narrower subset of language constructs.

Table 4.5: Complexity results per Author

Metric	Human	DSC	Qwen	Agg.
NLOC	26.41	6.84	10.66	14.38
CCN	5.97	1.89	2.79	3.50
TOKEN_COUNT	253.49	69.76	97.00	137.59
FUNCTION_NAME_LENGTH	18.33	15.07	17.63	16.99
Tot Functions	280405	298044	302657	881106
Unique Tokens (Totale)	60120	50195	50360	64706

Chapter 5

Conclusion

This section provides a high-level summary of the comparative analysis between human-written code and the code generated by the DSC and Qwen Large Language Models (LLMs).

Code Defects. In terms of general code defects, the analysis revealed that the human-written code, while syntactically correct, contained the highest overall volume of defects (1,006,282) and the highest average number of defects per sample (4.59). Both LLM-generated datasets showed a lower density of defects. DSC achieved the lowest average defects per sample (3.06), while Qwen produced the lowest percentage of defective files (74.04%). A significant trade-off was observed, as this performance was counterbalanced by a reliability issue not present in the human baseline: both models produced a substantial number of empty or not analyzable files (13,972 for DSC and 18,541 for Qwen). This suggests that while the generated code may be "cleaner"

on average, its generation is less reliable.

Security Vulnerabilities Regarding security vulnerabilities, the Qwen model demonstrated the strongest performance. It generated the lowest total number of issues (36,973) and the lowest percentage of vulnerable functions (6.72%). In contrast, the DSC model produced the most insecure code, with the highest rate of vulnerable functions (9.11%). Human-written code was positioned between the two models, with 8.90% of its functions found to be vulnerable. This indicates a clear advantage for the Qwen model in producing more secure code, although all authors (human and AI) were susceptible to high-severity vulnerabilities.

Code Complexity The most significant divergence was found in code complexity. Human-written code was, by a substantial margin, dramatically more complex across every metric measured. It was significantly longer (average NLOC of 26.41), possessed more intricate logical paths (average CCN of 5.97), and utilized a much richer lexical "vocabulary" (60,120 unique tokens). Conversely, the AI models produced far simpler and shorter code. DSC generated the simplest code (6.84 NLOC, 1.89 CCN), while Qwen's output was intermediate, though still vastly less complex than the human baseline. Both models also used a smaller, less diverse set of unique tokens (approx. 50,000). In summary, this analysis indicates that while human code is far more complex and lexically rich, it also carries the highest density of general

code defects. The LLMs, particularly Qwen, show a distinct advantage in generating code with fewer security vulnerabilities. However, the AI-generated code is structurally simplistic and currently suffers from reliability issues, evidenced by the high volume of empty or not analyzable outputs.

5.1 Future Work

Considering the proposed methodology, it would surely be interesting to extend this type of static analysis also to other languages. This work in fact is an example of it, as it extends the work of Cotroneo et al. [3] carried out previously on the programming languages Python and Java. Another path surely to undertake is surely that which includes the dynamic analysis of AI-generated code. To this end it is noted that, taking the state-of-the-art tools, it would be necessary to modify the dataset including not only code snippets but proper entire modules or even entire projects, as such tools of dynamic analysis have a necessity to know the context within which to work in order to produce significant tests.

Bibliography

[1] Amazon. Codewhisperer. `https://docs.aws.amazon.com/codewhisperer/`.

[2] Ram Chillarege et al. Orthogonal defect classification. *Handbook of software reliability engineering*, pages 359–399, 1996.

[3] Domenico Cotroneo, Cristina Improta, and Pietro Liguori. Human-written vs. ai-generated code: A large-scale study of defects, vulnerabilities, and complexity. *arXiv preprint arXiv:2508.21634*, 2025.

[4] Domenico Cotroneo, Cristina Improta, Pietro Liguori, and Roberto Natella. Vulnerabilities in ai code generators: Exploring targeted data poisoning attacks. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*, pages 280–292, 2024.

[5] Neil A Ernst and Gabriele Bavota. Ai-driven development is here: Should you worry? *IEEE Software*, 39(2):106–110, 2022.

[6] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*, 2020.

[7] Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu, and Jinfu Chen. Security weaknesses of copilot-generated code in github projects: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 34(8):1–34, 2025.

[8] GitHub. Github copilot. `https://github.com/features/copilot`.

[9] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. Deepseek-coder: When the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

[10] hauntsaninja. Tiktoken. `https://github.com/openai/tiktoken/tree/main/tiktoken`.

[11] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

[12] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436*, 2019.

[13] Cristina Improta, Rosalia Tufano, Pietro Liguori, Domenico Cotroneo, and Gabriele Bavota. Quality in, quality out: Investigating training data's role in ai code generation. *arXiv preprint arXiv:2503.11402*, 2025.

[14] Samia Kabir, David N Udo-Imeh, Bonan Kou, and Tianyi Zhang. Is stack overflow obsolete? an empirical study of the characteristics of chatgpt answers to stack overflow questions. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, pages 1–17, 2024.

[15] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, et al. The stack: 3 tb of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022.

[16] Yue Liu, Thanh Le-Cong, Ratnadira Widyasari, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D Le, and David Lo. Refining chatgpt-generated code: Characterizing and mitigating code

quality issues. *ACM Transactions on Software Engineering and Methodology*, 33(5):1–26, 2024.

[17] Logilab. Pylint- python linter. `https://www.pylint.org`.

[18] Michael R Lyu, Baishakhi Ray, Abhik Roychoudhury, Shin Hwei Tan, and Patanamon Thongtanunam. Automatic programming: Large language models and beyond. *ACM Transactions on Software Engineering and Methodology*, 34(5):1–33, 2025.

[19] Dung Nguyen Manh, Nam Le Hai, Anh TV Dau, Anh Minh Nguyen, Khanh Nghiem, Jin Guo, and Nghi DQ Bui. The vault: A comprehensive multilingual dataset for advancing code understanding and generation. *arXiv preprint arXiv:2305.06156*, 2023.

[20] Noble Saji Mathews and Meiyappan Nagappan. Test-driven development and llm-based code generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 1583–1594, 2024.

[21] Mitre. Mitre corporation. `https://www.mitre.org/`.

[22] mpokrass. Humaneval: Hand-written evaluation set. `https://github.com/openai/human-eval.git`.

[23] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. Codegen: An

open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*, 2022.

[24] OpenAI. Chatgpt. `https://chatgpt.com`.

[25] OpenAI. gpt-4. `https://openai.com/index/gpt-4/`.

[26] PMD. Source code analyzer. `https://pmd.github.io`.

[27] Semgrep. Static code analyzer. `https://github.com/semgrep/semgrep`.

[28] Bo Shen, Jiaxin Zhang, Taihong Chen, Daoguang Zan, Bing Geng, An Fu, Muhan Zeng, Ailun Yu, Jichuan Ji, Jingyang Zhao, Yuenan Guo, and Qianxiang Wang. Pangu-coder2: Boosting large language models for code with ranking feedback, 2023.

[29] SonarSource. Sonarqube. `https://docs.sonarsource.com/`.

[30] Rafael Sá Menezes, Mohannad Aldughaim, Bruno Farias, Xianzhiyu Li, Edoardo Manino, Fedor Shmarov, Kunjian Song, Franz Brauße, Mikhail R. Gadelha, Norbert Tihanyi, Konstantin Korovin, and Lucas C. Cordeiro. ESBMC: 7.4: Harnessing the Power of Intervals.

[31] terryyin. Lizard. `https://github.com/terryyin/lizard.git`.

[32] The Clang Team. clang-tidy v.21.1.0. https://releases.llvm.org/21.1.0/tools/clang/tools/extra/docs/clang-tidy/index.html.

[33] Norbert Tihanyi, Tamas Bisztray, Mohamed Amine Ferrag, Ridhi Jain, and Lucas C Cordeiro. How secure is ai-generated code: A large-scale comparison of large language models. *Empirical Software Engineering*, 30(2):47, 2025.

[34] vLLM. vllm. https://docs.vllm.ai/en/latest/.

[35] Yue Wang, Hung Le, Akhilesh Gotmare, Nghi Bui, Junnan Li, and Steven Hoi. Codet5+: Open code large language models for code understanding and generation. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 1069–1088, 2023.

[36] Xiaodan Xu, Chao Ni, Xinrong Guo, Shaoxuan Liu, Xiaoya Wang, Kui Liu, and Xiaohu Yang. Distinguishing llm-generated from human-written code by contrastive learning. *ACM Transactions on Software Engineering and Methodology*, 34(4):1–31, 2025.

[37] Burak Yetiştiren, Işık Özsoy, Miray Ayerdem, and Eray Tüzün. Evaluating the code quality of ai-assisted code generation tools: An empirical study on github copilot, amazon codewhisperer, and chatgpt. *arXiv preprint arXiv:2304.10778*, 2023.

[38] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, pages 1–12, 2024.

[39] yuriisk. clang-tidy-converter. `https://github.com/yuriisk/clang-tidy-converter.git`.